

CURSO TÉCNICO SUPERIOR PROFISSIONAL DE PROGRAMAÇÃO DE SISTEMAS DE INFORMAÇÃO
DESENVOLVIMENTO DE APLICAÇÕES

CONCEITOS BÁSICOS DA LINGUAGEM C#

Aula 1
Ano Letivo 25/26

C VS C#

C	C#
Procedimental (Sequência de Instruções organizadas em Funções)	Orientada a objectos
Multiplataformas	Dependente da .Net Framework
Instruções delimitadas por chavetas ({}) e cada linha termina com ;	
Estruturas de controlo - igual (if, swith)	
Estruturas de repetição - igual (for, while)	
Construção e utilização de funções e parâmetros – igual	
Criação e utilização de variáveis – igual	
Utiliza Structs	Utiliza Structs e Classes (Objetos)
São as duas fortemente tipadas	

ESTRUTURA DE UMA APLICAÇÃO WINFORMS

Program.cs (Início da Aplicação)

```
using System;  
using System.Windows.Forms;
```

Namespaces de referência a bibliotecas externas

```
namespace EscolhaFacil  
{
```

Namespace da aplicação (agregador de todas as classes, funções e variáveis da nossa aplicação)

```
    static class Program  
    {  
        /// <summary>  
        /// The main entry point for the application.  
        /// </summary>  
        [STAThread]  
        static void Main()  
        {  
            Application.EnableVisualStyles();  
            Application.SetCompatibleTextRenderingDefault(false);  
            Application.Run(new MainForm());  
        }  
    }  
}
```

Classe e função principal (inicial)

Criação do primeiro objeto (principal) que irá criar todos os outros (a aplicação termina quando este objeto morre)

MainForm.cs

```
using System.Windows.Forms;
```

```
namespace EscolhaFacil  
{  
    3 referências  
    public partial class MainForm : Form  
    {  
        1 referência  
        public MainForm()  
        {  
            InitializeComponent();  
        }  
    }  
}
```

MainForm.designer.cs

Outras Classes/Objetos (Mesmo Namespace)

```
ClasseCarro.cs  
ClasseCliente.cs  
...  
FormCarros.cs  
FormClientes.cs  
...
```

CONSTANTES E VARIÁVEIS

Constantes - Podem ser locais ou globais

```
public const int MaxTentativas = 5;
```

Variáveis

```
int idade = 25; // Declaração com inicialização  
string nome;    // Apenas declaração  
nome = "Ana";   // Atribuição de valor posterior
```

- Tipos de dados comuns: int, decimal ou double, char, bool, string...
- Podemos criar variáveis do tipo de uma classe que passam a ser objetos depois de instanciadas

FUNÇÕES

- Funções são métodos de uma classe.
- Podem ser executadas de forma estática sem existir um objeto ou apenas depois de um objeto criado.
- Podem conter ou não parâmetros de entrada de qualquer tipo
- Podem retornar qualquer tipo existente em C#

```
public class Carro
{
    public string Marca;
    public int Velocidade = 0;

    // Esta é a função (método)
    public void Acelerar(int incremento)
    {
        Velocidade += incremento;
    }
}
```

FUNÇÕES

Sem retorno (void) não necessita de **return**

```
public void Acelerar(int incremento)
{
    Velocidade += incremento;
}
```

Com retorno – obrigatório a existência de um **return** do mesmo tipo

```
public int Acelerar(int incremento)
{
    Velocidade += incremento;
    return Velocidade;
}
```

Sem parâmetros

```
public int Acelerar()
{
    Velocidade += 1;
    return Velocidade;
}
```

INSTRUÇÕES DE SELEÇÃO E DECISÃO

Instrução de decisão if...else – Decide o que vai ser executado

```
if (velocidade > 120) // Condição Booleana
{
    Console.WriteLine("Excesso de velocidade!");
}
else if (velocidade < 40)
{
    Console.WriteLine("Velocidade demasiado baixa.");
}
else
{
    Console.WriteLine("Velocidade dentro do limite legal.");
}
```

- O **else** não é obrigatório.
- As chavetas são obrigatórias sempre que se tenha mais que uma linha de código.
- Caso o resultado da condição seja verdadeiro, executa o que está dentro do **if**, caso seja falso, executa o que está dentro do **else**.

INSTRUÇÕES DE SELEÇÃO E DECISÃO

Instrução de seleção switch

```
switch (estadoSemaforo)
{
    case "Verde":
        Console.WriteLine("Pode avançar.");
        break;
    case "Amarelo":
        Console.WriteLine("Prepare-se para parar.");
        break;
    case "Vermelho":
        Console.WriteLine("Pare imediatamente.");
        break;
    default:
        Console.WriteLine("Sinal avariado.");
        break;
}
```

- O **break** é obrigatório para evitar a execução das instruções dentro do **case** seguinte, mas não é obrigatório no último **case** ou no **default**.
- O **case** valida o valor devolvido pelo **switch**, não permitindo o uso de condições.
- Tipos permitidos: int, long, byte, short, string, char.

INSTRUÇÕES DE REPETIÇÃO

Instrução de repetição **for**

```
int[] quilometragemMensal = new int[12];  
  
for (int i = 0; i < 12; i++)  
{  
    // Atribui 100km extra a cada mês  
    quilometragemMensal[i] = i * 100;  
}
```

- Utilizado quando se sabe previamente o número de vezes que o ciclo vai ser executado, por exemplo, para todos os meses.
- Consoante a condição poderá não ser executado nenhuma vez

INSTRUÇÕES DE REPETIÇÃO

Instrução de repetição **while**

```
int bateriaPercentagem = 100;  
bool modoPoupancaAtivo = false;  
  
while (bateriaPercentagem > 20)  
{  
    bateriaPercentagem -= 5;  
}  
  
modoPoupancaAtivo = true;
```

- Utilizado quando não se sabe previamente o número de vezes que o ciclo vai ser executado.
- Consoante a condição poderá não ser executado nenhuma vez

INSTRUÇÕES DE REPETIÇÃO

Instrução de repetição **do...while**

```
int capacidadeMaxima = 50; // Litros
int nivelAtual = 45; // O carro já tem algum combustível

do
{
    nivelAtual++; // Adiciona 1 litro de cada vez

    // A lógica: "Abastece primeiro, verifica depois"

} while (nivelAtual < capacidadeMaxima);
// Neste ponto, o nível será exatamente 50.
```

- Utilizado quando não se sabe previamente o número de vezes que o ciclo vai ser executado.
- Executa pelo menos uma vez.

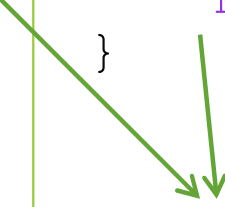
STRUCT VS CLASS

- Estruturas utilizadas para criar tipos de dados.
- Struct – tipo por valor, utilizada apenas para dados mais simples e casos pontuais.
- Class – tipo por referência, base da programação orientada a objetos, permite herança e suporta dados mais complexos.

```
public class Carro
{
    public string Modelo;
    public bool MotorLigado;

    public void Ligar()
    {
        MotorLigado = true;
    }
}
```

```
public struct PosicaoGPS
{
    public double Latitude;
    public double Longitude;
}
```



Atributos